

Git & GitHub

> **Ziel:**
>
> Diese Anleitung beschreibt standardisierte Abläufe für die Arbeit mit Git und GitHub in Verbindung mit VS Code. Sie umfasst die saubere Initialisierung von Repositories, die Anbindung an Remote-Repositories, den Einstieg in bestehende Projekte sowie zentrale Git-Konzepte und Befehle.

📁 Struktur

- 🔁 **[A] Typischer Workflow (für VS Code & Git)**
- 🧩 **[B] Neues Projekt aufsetzen**
- 👥 **[C] Bestehendem Projekt beitreten**
- 🧠 **[D] Git-Grundlagen**
- 🧨 [E] Git: Mergen & Konflikte (TODO)

Stand: [24.04.2025]

🔁 [A] Typischer Workflow (für VS Code & Git)

- **(0.) Auf den gewünschten Branch wechseln** (`git checkout <branch-name>`)
  - → Vor dem Coden immer sicherstellen, dass man im richtigen Branch ist.
  - → Falls nötig, vorherige Änderungen committen oder stashen (`git stash`), da der Checkout sonst fehlschlagen kann.
  - Optional: Aktuellen Stand vom Remote holen
    - Vorsichtig (nur abrufen): `git fetch`
    - Direkt mit lokalem Stand zusammenführen: `git pull`
- **(1.) Coden und Änderungen speichern** (Alle bearbeiteten oder neu erstellten Dateien mit `Strg + S` sichern).
- **(2.) Datei(en) zu Git hinzufügen**:
  - → vorher prüfen, ob man sich im richtigen Verzeichnis befindet (oder Pfad angeben)
  - Eine Datei hinzufügen: `git add <dateiname>`
  - Alle Änderungen im Projekt hinzufügen: `git add .`
    - → erfasst neue, geänderte und gelöschte Dateien im aktuellen Verzeichnis + Unterordnern
- **(3.) Commit erstellen:** `git commit -m "Kurze, sinnvolle Commit-Nachricht"`
- **(4.) Änderungen pushen:** `git push`
  - Nur beim **ersten Push eines neuen Branches**: `git push -u origin <branch-name>`
- **Weiteres Nützliches:**
  - Status prüfen (jederzeit): `git status`
  - Letzten Commit anzeigen (kurz): `git log -1`, oder `git log --oneline` → Praktisch zur Kontrolle vor dem Push.
  - Dateien wieder aus dem Staging entfernen (falls man zu viel mit `git add` ausgewählt hat): `git restore --staged <dateiname>`

- Änderungen vor dem Commit anschauen: `git diff` (noch nicht gestagte Änderungen) `git diff --staged` (bereit zum Commit)
- Temporär Änderungen speichern: `git stash` (Hilfreich, wenn man schnell den Branch wechseln muss, ohne zu committen.)
- Weiteres

🧩 [B] Neues Projekt aufsetzen

🟢 B.0 – Git installieren (falls noch nicht geschehen)

📌 **Szenario:**

Git ist auf dem System noch nicht installiert. Vor der Arbeit mit einem Repository muss Git eingerichtet und mit dem eigenen GitHub-Konto verbunden werden.

🧭 **Ablauf:**

Nach der Installation werden grundlegende Nutzerdaten konfiguriert, um spätere Commits korrekt zuzuordnen. Zusätzlich werden Hinweise zur Nutzung der Git Bash gegeben.

🪜 **Schritte:**

1. Git installieren:

   🔗 https://git-scm.com/downloads

   Während der Installation:

   - Als Editor sollte **nicht VIM**, sondern z. B. **VS Code** ausgewählt werden.
   - Alle weiteren Voreinstellungen können in der Regel unverändert übernommen werden.

2. Git global konfigurieren (einmalig):

   Diese Angaben werden für alle zukünftigen Commits verwendet:

```
git config --global user.name "Git Username"
git config --global user.email "Git adresse@mail.de"
```

3. Git nutzen: Git kann dann entweder über **Git Bash** oder auch einfach über das integrierte **Terminal in VS Code** verwendet werden.

⚠️ **Hinweise:**

- Nützliche Tastenkombinationen für **Git Bash**:
  - Einfügen von kopierten Befehlen: `Shift` + `Insert`
  - Alternativ aktivierbar: `Ctrl + Shift + V`

    → Rechtsklick → *Options…* → *Keys* → Haken setzen bei „Ctrl+Shift+letter shortcuts“
- Die aktuelle Git-Version kann jederzeit mit `git --version` überprüft werden.
- Die Konfiguration mittels `user.name` und `user.email` ist global gültig – sie kann jedoch bei Bedarf auch repository-spezifisch überschrieben werden.

🟢 B.1 – Lokaler Projektordner existiert bereits

📌 **Szenario:** Ein Projekt wurde lokal begonnen. Die Dateien liegen bereits in einem Ordner auf dem Rechner, aber Git wurde noch nicht eingerichtet. Ziel ist es, den bestehenden Ordner mit einem Repository auf GitHub zu verbinden.

🧭 **Ablauf:** Der Ordner wird zu einem Git-Repository gemacht ("initialisiert") und mit einem GitHub-Repo verknüpft. Falls das GitHub-Repo bereits Dateien enthält (z. B. `README.md`), müssen beide Stände zusammengeführt werden.

🪜 **Schritte:**

1. Git initialisieren:

```
git init
```

2. Remote-Repository verknüpfen:

```
git remote add origin https://github.com/NUTZERNAME/REPO-NAME.git
```

3. Falls GitHub bereits Dateien enthält:

```
git pull origin main --allow-unrelated-histories
```

4. Konflikte (falls vorhanden) manuell lösen, dann:

```
git add .
```

```
git commit -m "Initial commit with local files"
```

→ Achtung: `git add .` fügt **alle Dateien** im aktuellen Verzeichnis und allen Unterordnern hinzu!

5. Änderungen hochladen:

```
git push -u origin main
```

Hinweise:

- Bei Konflikten zeigt Git die betroffenen Dateien an.
- `.gitignore` und `README.md` können doppelt vorkommen.
- `-u` verbindet Branch dauerhaft mit Remote.
- Darauf achten, `git init` im richtigen Ordner auszuführen.

🟢 B.2 – Neues Projekt ohne bestehende Dateien starten

📌 **Szenario:** Es existieren noch keine lokalen Dateien. Ein neues Projekt soll von Grund auf sauber aufgebaut und mit GitHub verbunden werden.

🧭 **Technischer Ablauf:** Ein leerer Projektordner wird lokal erstellt und als Git-Repository initialisiert. Je nach Vorgehensweise kann das Repository zuerst auf GitHub oder zuerst lokal angelegt werden. Beide Varianten sind möglich.

🪜 **Variante 1 – Zuerst GitHub-Repository erstellen:**

1. Neues Repository auf GitHub erstellen (ohne README, falls du lokal eins anlegen willst)
2. Lokalen Projektordner anlegen
3. Im Ordner Git initialisieren:

```
git init
```

4. Optional: `README.md`, `.gitignore`, `LICENSE` lokal anlegen
5. Remote hinzufügen:

```
git remote add origin https://github.com/NUTZERNAME/REPO-NAME.git
```

6. Änderungen committen:

```
git add .
git commit -m "Initial commit"
```

7. Push zu GitHub:

```
git push -u origin main
```

🪜 **Variante 2 – Zuerst lokal starten:**

1. Projektordner lokal anlegen
2. Dateien wie `README.md`, `.gitignore`, `LICENSE` hinzufügen
3. Git initialisieren:

```
git init
```

4. Repository auf GitHub erstellen (ohne README, um Konflikte zu vermeiden)
5. Remote hinzufügen:

```
git remote add origin https://github.com/NUTZERNAME/REPO-NAME.git
```

6. Änderungen committen und pushen wie oben

⚠️ **Hinweise & Stolperfallen:**

- Falls GitHub bereits Dateien enthält, beim ersten Pull `-allow-unrelated-histories` nutzen.
- `.gitignore` frühzeitig anlegen, um unnötige Dateien auszuschließen.
- Nach dem Push kann das Projekt auf GitHub mit anderen geteilt oder es können Branches eingerichtet werden.

👥 [C] Bestehendem Projekt beitreten

🔵 C.0 – Git installieren (falls noch nicht geschehen)

→ siehe B.0

🔵 C.1 – Bestehendes GitHub-Repository klonen

📌 Szenario:

Ein bestehendes GitHub-Repository soll lokal geklont und mit VS Code geöffnet werden, um daran weiterzuarbeiten.

🧭 Ablauf:

Ein neuer Projektordner wird lokal erstellt und als VS Code-Workspace geöffnet. Danach wird das Repository via Terminal in diesen Ordner geklont.

🪜 Schritte:

1. Lokalen Zielordner erstellen (dieser dient als Container für das Projekt).
2. Ordner mit Rechtsklick → **„Mit Code öffnen“** (oder VS Code starten und manuell öffnen).
3. In VS Code ein neues Terminal öffnen (Tastenkürzel: `Strg + ö` oder über Menü: Terminal → Neues Terminal).
4. Im geöffneten Terminal ausführen:

```
git clone https://github.com/NUTZERNAME/REPOSITORY-NAME.git
```

Bspw. für StoryFinder-v2:

```
git clone https://github.com/RaphaelNoah/storyfinder-v2.git
```

5. Der geklonte Projektordner erscheint nun im Dateibaum von VS Code.
6. Optional: Geklonte Unterordner als neuen Workspace öffnen (rechte Maustaste → „Ordner im neuen Fenster öffnen“).

⚠️ Hinweise & Tipps:

- Git lädt den vollständigen Projektstand vom Remote-Repository (inkl. History).
- Standardmäßig wird automatisch der `main` Branch ausgecheckt.
- Danach können neue Branches erstellt oder bestehende Branches (z. B. `dev`) aktiviert werden (mit `git checkout`, `git checkout -b`, etc.).
- Mehr zum Umgang mit Git: siehe 🔁 **[A] Typischer Workflow (für VS Code & Git) und** 📘 **[D] Git-Grundlagen**

🔵 C.2 – Lokale Dateien sollen mit bestehendem GitHub-Repository verbunden werden

📌 Szenario:

Ein lokaler Projektordner mit bestehenden Dateien soll nachträglich mit einem bereits existierenden GitHub-Repository verbunden werden.

🧭 Ablauf:

Der bestehende Ordner wird lokal zu einem Git-Repository gemacht. Danach wird das Remote-Repository verknüpft, und beide Stände werden ggf. zusammengeführt.

🪜 Schritte:

1. Lokalen Projektordner in VS Code öffnen (Rechtsklick auf den Ordner → „Mit Code öffnen“).
2. Terminal in VS Code öffnen (`Strg + ö` oder über Menü).
3. Git initialisieren:

```
git init
```

4. Remote hinzufügen:

```
git remote add origin https://github.com/NUTZERNAME/REPOSITORY-NAME.git
```

5. Inhalte vom Remote-Repository abrufen (nur wenn dort schon Dateien vorhanden sind):

```
git pull origin main --allow-unrelated-histories
```

6. Konflikte (falls vorhanden) manuell lösen, dann Änderungen vorbereiten:

```
git add .
git commit -m "Initial commit mit lokalen Dateien"
```

7. Änderungen hochladen:

```
git push -u origin main
```

⚠️ **Hinweise & Tipps:**

- `-allow-unrelated-histories` wird nur beim ersten Pull benötigt, wenn GitHub bereits Dateien enthält.
- Darauf achten, dass man im richtigen Projektordner arbeitet.
- `.gitignore` nicht vergessen, um z. B. `.vscode/`, `.venv/` oder temporäre Dateien auszuschließen.

📘 [D] Git-Grundlagen

🧠 D.1 – Verständnis: Wie Git funktioniert

🗃️ Lokales vs. Remote-Repository

- Das **lokale Repository** ist die Git-Verwaltung auf dem eigenen Rechner. Es enthält:
  - die Projektdateien,
  - eine Historie der Änderungen (Commits),
  - Informationen zu Branches.
- Ein **Remote-Repository** (z. B. auf GitHub) ist eine online gehostete Kopie des Projekts.
- Die Verbindung erfolgt über einen Remote-Namen – **standardmäßig** `origin`.
- Mit `git remote -v` lassen sich die aktuell verknüpften Remotes anzeigen.

🌿 Was ist ein Branch?

- Ein **Branch** ist eine eigene Entwicklungslinie – nützlich für neue Features, Bugfixes, Tests etc.
- Branches ermöglichen paralleles Arbeiten, ohne die Hauptversion (`main`) zu verändern.
- Beispielhafte Branch-Typen:
  - `main` → stabile Hauptversion
  - `dev` → aktiver Entwicklungsstand

- `feature/<name>` → neue Funktion
- `sandbox/<name>` → Experimente oder Tests

- Ein Branch kann **lokal** (nur auf dem eigenen Rechner) oder **remote** (auf GitHub) existieren.

🛠️ D.2 – Mit Git arbeiten: Wichtige Befehle & Abläufe

- Für typischen Workflow mit Git, 🔁 **[A] Typischer Workflow (für VS Code & Git)**

🔍 Status prüfen & Verlauf anzeigen

- `git status` – zeigt den aktuellen Zustand (geänderte Dateien, aktiver Branch etc.)
- `git log` – vollständige Commit-Historie
- `git log --oneline` – kompakte Übersicht über vergangene Commits
- `git show` – Details zum letzten Commit anzeigen

📦 Dateien zum Commit vorbereiten (Staging)

- Einzelne Datei zum Staging hinzufügen: `git add <dateiname>`
- Alle Änderungen im Projekt hinzufügen: `git add .`

💾 Commit erstellen

- Commit mit Nachricht speichern:
  `git commit -m "Kurze, sinnvolle Commit-Nachricht"`

📤 Änderungen hochladen

- Bereits verbundener Branch: `git push`
- Neuer Branch (beim ersten Push): `git push -u origin <branch-name>`

📥 Änderungen herunterladen

- `git pull` – zieht aktuelle Änderungen vom Remote und integriert sie
- `git fetch` – holt Änderungen, ohne sie direkt zusammenzuführen (manuelles Mergen möglich)

🌿 Mit Branches arbeiten

- Aktuellen Branch anzeigen: `git branch` (aktiver Branch ist mit markiert)
- Alle Branches inkl. Remote anzeigen: `git branch -a`
- Neuen Branch erstellen und direkt wechseln: `git checkout -b <neuer-branchname>`

  → Der neue Branch basiert auf dem **aktuellen Branch**, daher vorher prüfen, auf welchem man sich gerade befindet.
- Remote-Branch lokal auschecken: `git checkout -b <lokal> origin/<remote>`

  z. B. `git checkout -b dev origin/dev`

  → Erstellt einen lokalen Branch auf Basis eines vorhandenen Remote-Branches
- Zu einem bestehenden Branch wechseln: `git checkout <branchname>`

🚫 Dateien ignorieren mit `.gitignore`

- Nicht benötigte Dateien (z. B. `.vscode/`, `.venv/`, `__pycache__/`) sollten ausgeschlossen werden
- `.gitignore` im Hauptverzeichnis platzieren
- Vorlagen z. B. unter: https://www.toptal.com/developers/gitignore

---

↩️ Änderungen rückgängig machen

- Aus Staging entfernen: `git restore --staged <dateiname>`
- Änderungen an Datei verwerfen: `git restore <dateiname>`
- Letzten Commit zurücknehmen, aber Änderungen behalten: `git reset --soft HEAD~1`

📦 Temporäre Änderungen sichern (stash)

- Zwischenspeichern: `git stash`
- Wiederherstellen: `git stash pop`
- Anzeigen: `git stash list`
- Entfernen: `git stash drop`

🧨 [E] Git: Mergen & Konflikte (TODO)

📌 Ziel:

Dieser Abschnitt erklärt, wie Änderungen aus verschiedenen Branches

zusammengeführt ("gemerged") werden und was passiert, wenn es dabei zu Konflikten kommt.